Hi! The following underlines the steps I took to get this dashboard working, the data I used, the analysis tools I used and most importantly - what the linguistic metrics mean.

First of all - where is this idea from? Basically - there's this foreign language acquisition theory (made by Stephen Krashen), which essentially boils down to the statement that a relaxed learner, consuming content that's both slightly above their current level (i+1, where i is their current level) and that they're interested in, will naturally acquire the language they're learning (Krashen, 1984). In one of the YT videos of his lectures Krashen was asked how video games play into this theory - and he said that that's an interesting thing to be researched. So I researched it, carrying out a pilot study (with only 60 participants) who were asked about their experiences with video games in English language acquisition. A statistical analysis of their responses showed that, in their opinion, the medium of video games was compatible with Krashen's theory (Niewiarowski, 2026, in press). Now, I would take that with like twenty grains of salt, seeing as it was a pilot study on such a small sample, but that still got me thinking.

If learners acquire consuming content slightly above their level, and video games are compatible with this theory, then how do learners know if something is slightly above their level? There was no tool to check it, nothing of that sort. So, this is where this dashboard comes in. Now, I know what you're thinking: "How did you get the dialogues for this??? Aren't games, like, closed-source? And also encrypted?"

Yes. Yes, they are. But I managed to find an incredible github repo which made it all possible. The Video Game Dialogue Corpus (https://github.com/seannyD/VideoGameDialogueCorpusPublic) (Rennick & Roberts, in press; Rennick et al, 2023) is a corpus that allows you to get dialogue data from over 50 popular (at least some of them) games, ranging all the way from series like Ace Attorney and Persona to The Witcher. That corpus takes care of sourcing the dialogue data, parsing it into neat .json files, all ready for analysis!

The main issue was that this corpus was made years ago, and using it the way it was intended simply did not work. Some of the sources no longer existed and had to be scraped from the Internet Archive (bless you, archivists). Some of them were behind Cloudflare protection and had to be downloaded manually (like some 14 pages for each of the three Ace Attorney games). And, as you might imagine, when you scrape data from the Internet Archive, or generally places different from the initial creators of the corpus - the scraper doesn't work either. So the scraper had to be fixed for many of these games.

But, once I got everything working and the 56 games' dialogue data was all ready to analyse, I started doing just that. The entire analysis pipeline is available at https://github.com/Kamillon125/the_language_of_video_games, however I do believe it is crucial to describe this entire analysis process in depth - this is what the next section is for.

STEP 1: EXTRACTION

The people making the Video Game Dialogue Corpus included useful tags before every text line in the game, such as LOCATION, METADATA, ACTION, NARRATIVE, STATUS or SYSTEM. Additionally, there were other metadata tags, all of them starting with the underscore _. What I did, was completely ignore all these tags and only source the ones representing character names, which were the actual dialogue data. Those dialogues were saved to game_dialogue.txt, which was the file that was later being analysed. An important note here, however, is that, as you might be able to see,

there are two Final Fantasy XIV games. That's for the simple reason that the parser did not work that well here, hiding quite a lot of the game's dialogue behind the "ACTION" tag (which represents out-of-game-world notes from, for example, walkthrough makers, if the data was sourced from a walkthrough). This is why, for this game specifically, I included two versions in the dashboard - one with JUST the actual dialogues, which is the slimmer version with less text, and one with both the dialogues AND all the ACTION data, which also sometimes included dialogues.

STEP 2: spaCy

spaCy (https://spacy.io/) is a Python library which made this entire project possible - it was the workhorse of the language analysis used here. That is why I am reserving an entire step in this methodology just for spaCy. It was used, for instance, to get a standard English language "vocabulary" in the form of its "en_core_web_md" model, that was used for vocab analysis. It was also used, most notably, for the function "nlp", which turned the entire grammatical text into data that was processable by Python. It assigned each word its role in the sentence, its part of speech. It also made it very easy for me to split the dialogue data into sentences or lemmatise words (turn them into their base forms for analysis), and so much more.

STEP 3: BASIC METRICS

There's a few metrics in linguistics, which can be used to analyse text (Eleyan et al, 2020; Zhou et al, 2017; Dale & Chall, 1948; Dale & Chall, 1995). Using the Textatistic python package (https://pypi.org/project/textatistic/) I calculated four of these metrics - the Flesch Reading Ease, the Flesch-Kincaid Grade, the SMOG Index and the Dale-Chall Score. Additionally I calculated the amount of total words analysed.

After that I used spaCy's nlp() to tokenise the text, tag the parts of speech used and lemmatise the text. For each word that wasn't a stop-word, punctuation or space, and also was not a proper noun, interjection, symbol, number or an unknown word that spaCy had no idea how to classify, I appended it to the meaningful words list, returning the top 20 later. That was a rather restrictive process, but I had to do all that to be assured I minimised the odds of a random, meaningless word appearing in the top 20 list. Using the lexical_diversity python package (https://pypi.org/project/lexical-diversity/) I calculated the TTR and MTLD of non-punctuation and non-space tokens.

I then used spaCy to split the text into sentences, calculate the total words and sentence amount, and get the average and median sentence lengths using python's statistics module (https://docs.python.org/3/library/statistics.html).

What was left in this section was to calculate the rate of Out-of-Vocabulary words and the most popular bigrams. An OOV word is a token that appeared in the game's dialogue, but was NOT in the english "dictionary" used by spaCy. A bigram is a combination of two words appearing right after one another (eg: "fast dog", "I am", "you are").

STEP 4: CEFR WORD ANALYSIS

This is where the meat and potatoes of this project are. The way this worked in the project was that, utilising spaCy's tags, entity filters and my own rudimentary slang filter (available at cefr_words.py in the github repo) I fed all the lemmatised words in the text through the CEFRSpaCyAnalyzer, made available in the cefrpy python module (https://pypi.org/project/cefrpy/), which assigned a CEFR (Figueras, 2012) level to each and every single word. After this, it checked every single lemmatised word for the lowest CEFR level it appears at and "flattened" the words to the lowest level (this is so that, for example, "point" is not both in B2 as a verb and in B1 as a noun"). CEFRSpaCyAnalyzer assigned each word to either a CEFR level, or UNKNOWN (represented on this dashboard as Unclassified in the 2B section). Afterwards, this script, for each CEFR level, got the total and unique

amount of words, and their respective ratio to all the words in the text.
This data is extremely important, and I'll spend the next few sentences explaining why.
First of all, Laufer stated in 1992, that in order to comfortably understand a text, you have to understand 95% of the words in this text. After that, Hu & Nation in 2000 said, that to acquire from a text, you must understand 98% of the vocabulary. This allowed me to calculate what CEFR level you'd have to be at to reasonably understand and acquire from a given game, both in terms of its total vocabulary and unique vocabulary. It stands to reason, however, that these thresholds might be a bit lower in video games, due to them being an interactive audiovisual medium (Wolf, 2002) that helps in understanding words with the context it provides (Shahiwala & Rahul, 2025).
Secondly, equally importantly, Nation stated in 2014, that in order to acquire a word, a learner should be exposed to it 12 times, which allows the word to appear in a variety of forms and contexts. This is another thing that was extremely important in this project, to the point that it appears right at the very top in the "Learning Utility" section. What I did here was to count the amount of words that appear 12 times or more for each CEFR level and display it on a bar chart. Perhaps even more importantly, however, I attempted standardising this metric in the "Acquisitional Index", which is the ratio of lemmas (base form words) that appear 12 or more times, to all the lemmas in the text at a given level. Additionally, a list of those lemmas is available below the bar charts to show learners what words they could reasonably expect to acquire after playing through a given game - another extremely important part of this dashboard. The acquisitional index definitely needs to be tested more thoroughly in other research.

STEP 5: AI SENTENCE CLASSIFICATION
After splitting the text into sentences, I loaded up the "dksysd/cefr-classifier" model (https://huggingface.co/dksysd/cefr-classifier), which is fine-tuned from microsoft/deberta-v3-large. It classifies sentences on the CEFR level scale, with an 89.7% accuracy across a parallel corpus dataset. I ran this model on my GPU in batches of 64 sentences in parallel using the Hugging Face pipeline API (with the Dataset structure). If I ran these sentences one-by-one, it would have taken me many days to analyse all 1 333 761 sentences. Additionally, if a sentence was over 512 tokens long, it got cut off, seeing as normally sentences are NOT over 500 tokens long and that would have been a result of a parsing mistake. After doing all that, I returned a neat csv with the sentence amounts (and percentages) for each CEFR level, while also saving that to a json.

STEP 6: ADVANCED METRICS
Once again, using the slang filter, spaCy's nlp() and spaCy's sentence splitter, I calculated the ratio of passive voice per sentence, subordinate clauses per sentence, conditionals per sentence, slang density and phrasal verb density per 1000 words. Here I made use of spaCy's dep_ property, which told me the role of a token in a sentence. This way I could isolate, for example, tokens introducing passive voice into a sentence (auxpass, nsubjpass), or tokens introducing subordinate clauses (mark, advcl). To find the conditionals, i checked how many "if" tokens whose role was "mark" were. To check the amount of phrasal verbs, I checked the amount of tokens marked as particles "prt", whose head was a verb. And to check the amount of collocations - in this case the combinations of verbs and prepositions and the combinations of nouns and adjectives, I simply checked the part of speech of a token, and then in its children I checked if there was a preposition or an adjective respectively.

STEP 7: LEXICAL DENSITY
Lexical density is the ratio of lexical items to all the tokens in the text. As per Johansson (2008), lexical items include nouns, verbs, adjectives and adverbs. However, I also wanted to know how much proper nouns (words like London or Toyota) impact the lexical density of any given game's dialogue,

so I filtered out two different lists. First, a list of all words that were nouns, adjectives, verbs and adverbs, and then also a list of all those words plus proper nouns. This way I got two different measures of lexical density in any given game - both including and not including proper nouns.

STEP 8: GETTING IT ALL TOGETHER
The last part of the project was a file getting all of this together. I used runpy (https://docs.python.org/3/library/runpy.html) to get one file to run different python files and get the global variables from them. On top of every .py file returning a .csv or a .txt with its own results, whether that be the sentences classified into CEFR levels, of the words classified into CEFR levels, this one also returned a .json with all the relevant data for each game. That data included both the metadata information, such as the game's name, the series' name, the year of publication, the source material for the dialogues and the source_features information - all from the Video Game Dialogue Corpus, but also things I gathered myself. Here is a list of all the data that was in every game's .json: total lines extracted, Flesch Reading Ease, Flesch-Kincaid Grade, SMOG Index, Dale-Chall score, word count, passive voice per sentence, subordinate clauses per sentence, conditionals per sentence, slang density percentage, phrasal verbs per 1000 words, top 20 lemmas, top 15 bigrams, top 15 verb-preposition collocations, top 15 adjective-noun collocations, top 15 phrasal verbs, TTR, MTLD, average sentence length, median sentence length, OOV words percentage in text, top 10 OOV words, full OOV list, lexical density including and not including proper nouns, proper noun density contribution, all the CEFR data for both words and sentences - including the full lists of B2, C1, and C2 learned vocab, the Acquisitional Index for every CEFR level, the comprehension and acquisition thresholds, the CEFR words distribution, etc. All this was saved into a .json with the following name structure: [game_name]_analysis_results.json, where game_name was the name of the game, as sourced from the meta.json file from the Video Game Dialogue Corpus. Additionally, I collected the information on whether a game has voiceover for every single game in the corpus, as the presence of both audio and visual information at the same time may boost comprehension, in line with the dual coding theory (Mayer & Sims, 1994).

STEP 9: COMBINING ALL THE GAMES
After getting the .json for every single game, I ran a simple python script to get the median score for every metric, which allowed me to also put in the "CORPUS MEDIAN" game data into this dashboard. I also ran another python script to combine all the .json files into "dashboard_data.json", which is the main file I sourced the data from, also available in this website's github repository (https://github.com/Kamillon125/kniew).

— HOW TO READ THIS DATA —

In this part I will go section by section, to fully describe how to read and interpret the data for any given game in the dashboard.

SECTION 0: METADATA
This is where you can see what the game's title is, the series it comes from, the year it released, etc. The entirety of this data (up to "Voiceover") was sourced entirely from the Video Game Dialogue Corpus. What is important to pay attention to here is the type, completeness and voiceover. Basically, the type is where the data was sourced from - the most accurate sourcing method is getting it directly from game files (like Stardew Valley), but the majority of these games were sourced from walkthroughs, which are also decently accurate. Completeness is how complete the game's dialogue

data is - ranging all the way from “sample” (only a sample of the game, the least accurate) to “complete”. Voiceover tells you whether the game is text-only in its dialogues, or if it’s also voiced. If it’s voiced, that might make it easier to comprehend and acquire, as that would be in line with the dual coding theory (Mayer & Sims, 1994). MOST IMPORTANTLY - the amount of lines extracted and the game’s completeness dictates how accurate the rest of the data is. If we’re dealing with an incredibly big game that only had a sample of its dialogues collected, the data here will be extremely inaccurate, especially for word counts. However, if we’re dealing with a small, text-based game, then naturally, even if the data is complete, it’s not going to have that many words. Basically, completeness is how much you can “trust” this data.

SECTION 1: Learning Utility

Firstly, let’s talk linguistics. As per Nation (2014), when a word appears 12 or more times, it can be considered learned, as the learner has had time to expose themselves to many forms and contexts that the word can be in. Whenever, in this entire website, a word is referred to as “learned”, that means it appeared 12 or more times. As such, here you can see two bar charts - the first one showing the game’s unique vocab (compared to the median), and the second one showing the game’s learned vocab (compared to the median of the corpus).

Below that, you can see the acquisitional indexes, which are my form of standardising this formula. They’re the amount of lemmas (base forms of a word) that appeared 12 or more times in a given game, for each CEFR level from B2 to C2. A1-B1 were omitted on purpose here, as they include very universal vocabulary that many games repeat, and as such would just introduce unnecessary noise. You can also see the full lists of the words appearing 12 or more times in a game for each CEFR level (Figueras, 2012).

SECTION 2A. Syntactic & Lexical Metrics

Here you’ve got some neat information about the language used in the game. Let’s go over each one of these and what they mean.

Phrasal verbs are traditionally understood as made up of a verb and an adverbial particle (Riguel, 2014). Some examples may include “make up”, “shut down”, “give up”. They often mean something different together than what you think they might mean on their own - “give up” meaning to stop trying, as opposed to taking something and putting it somewhere higher.

Slang density is the ratio of words detected by my own (limited) slang filter divided by all the words.

Lexical density is the ratio of lexical items to all the tokens in the text. As per Johansson (2008), lexical items include nouns, verbs, adjectives and adverbs. However, I also wanted to know how much proper nouns (words like London or Toyota) impact the lexical density of any given game’s dialogue, so I filtered out two different lists. First, a list of all words that were nouns, adjectives, verbs and adverbs, and then also a list of all those words plus proper nouns. This way I got two different measures of lexical density in any given game - both including and not including proper nouns.

Passive voice is easiest to illustrate in an example, as per https://developers.google.com/tech-writing/one/active-voice (accessed April 29, 2026). For example, passive voice is “the mat was sat on by the cat”, whereas active voice would be “the cat sat on the mat”.

Subordinate clauses, as per https://www.grammarly.com/blog/grammar/subordinate-clause/ (accessed May 5th, 2026), are clauses that cannot stand alone as a complete sentence and rely on the main clause for a meaning. An example here may be “the dog barked WHEN THE MAILMAN ARRIVED”, or “he read his book WHILE HE WAS WAITING”.

Conditional sentences, as per https://www.scribbr.com/verbs/conditional-sentences/ (May 5th, 2026) refer to a hypothetical situation and its possible consequences. They always contain a subordinate clause expressing a cnodition, for example: “I will wear a coat IF IT RAINS”.
TTR is Type to Token Ratio. It divides the amount of unique words (a unique word is a type) by all the words (a word is a token), though it generally does become lower the lkonger a text is (Richards, 1987), because if you have an extremely long book, then even very rare words start repeating and are no longer “unique”.
MTLD on the other hand is another measure for text diversity, but it is way less dependent on text length (McCarthy & Jarvis, 2010).

SECTION 2B. CEFR Level Distribution
This is a simple bar chart showing the amount of both total and unique words per CEFR level, also including unclassified words.

SECTION 3A. Thresholds
As you may be able to see, there are two different thresholds (95% and 98%), divided into both “total” and “unique”. According to Laufer (1992), you have to know 95% of the words in a text you’re reading to comprehend the text, and according to Hu & Nation (2000), you have to know 98% to comfortably acquire new words. That’s what the 95% and 98% are for, whereas the “total-unique” split is just to show what level you’d have to be at for each of these, if we were only talking about unique words (this is way higher, since more lower-level words repeat more often). It stands to reason, however, that these thresholds might be a bit lower in video games, due to them being an interactive audiovisual medium (Wolf, 2002) that helps in understanding words with the context it provides (Shahiwala & Rahul, 2025).

SECTION 3B. CEFR Sentences
This is the amount of sentences classified in each CEFR level by the AI model I was using, which you can read more about in the technical rundown in STEP 5: AI SENTENCE CLASSIFICATION.

SECTION 3C. Readability (L1)
These are various readability metrics
The Flesch Reading Ease tells you how easy a text is to read, the higher the easier (Eleyan et al, 2020). The Flesch-Kincaid Grade assigns the US school grade necessary to read a text (Eleyan et al, 2020). The SMOG Index assigns the text a difficulty rating based on the amount of polysyllabic words (3 or more syllables), also assigning the text an estimated amount of years of education needed to understand it (Zhou et al, 2017). The Dale-Chall score, however, bases its results on a list of words that American 4th graders could reliably understand, considering any word not on there difficult (Dale & Chall, 1948; Dale & Chall, 1995).
SECTION 4A. Top Counts
This section shows you the top 10 for lemmas, bigrams, collocations of verb-preposition and adjective-noun, and phrasal verbs. A lemma is the base form of a word, bigrams are combinations of two words, and the other three have been neatly described in SECTION 2A. Syntactic & Lexical Metrics.

SECTION 4B. OOV & Outliers
OOV stands for Out-Of-Vocabulary, and it refers to words which were not in the english dictionary I used during analysis. The OOV rate is the ratio of OOV words to the entire text’s length, Top 10 OOV

are the OOV words appearing the most often, and you can also toggle a full OOV list showing all the OOV words in a given game.

LIMITATIONS AND AREAS OF IMPROVEMENT

As with any project of this magnitude, especially one concerning natural language, there are several limitations to consider. The first, most important one, is definitely the absolute indescribable nature of human language - regardless of how much one wants to operationalize it, some nuances always remain undetected.

What's also noteworthy is that there is simply no feasible way of knowing exactly what words somebody knows and what words they don't know, which renders an accurate acquisitional index score quite literally impossible to calculate. This project aims to circumvent that by applying a standardised CEFR framework to its calculations, however that creates another limitation - it assumes that someone knows all the words in the B2 category if they're B2, and knows no C1 or C2 words. That is why, in addition to the acquisitional score, each game also shows you the actual vocabulary you will learn from it when playing.

Yet another limitation, albeit a less important one, is the difference in tokens and types between different dialects of English. For instance, in the case of Disco Elysium, "armour" was counted as a C2 word despite being rather common, especially in video games, likely because it's a British English way of spelling "armor". At this scale, when analysing millions of words, these differences are rather minute and likely don't impact the results much, however they should also be considered while interpreting the results.

Another limitation worth considering is the rather small amount of games analysed by this project, which is simultaneously the most noteworthy area of improvement. I analysed all the games I could in the Video Game Dialogue Corpus (as some of them I did not have access to, or the websites were not even available on Internet Archive), but a selection of 56 games is still not even a droplet in the ocean of all video games. Thankfully, most of these games are rather popular, which does mitigate this issue somewhat, but even so, the absence of extremely popular games like the Pokemon series is difficult to ignore.

Another methodological assumption, which may be a flaw, was flattening every word to its lowest CEFR level. That was done to consolidate all the lemmas, while assuming that if someone knows the word "point" as a verb, it's going to be way easier and almost intuitive for them to grasp the meaning of the noun "point", however that may not be the case with every example.

It is also worth noting that the classifier used for sentences is not 100% accurate, as it scored an 89.7% accuracy rating on a confusion matrix for its parallel corpus dataset - video game sentences are also inherently different from the training dataset for this model.

Video games also include branching dialogue, storylines and optional quests. Especially in RPGs like The Witcher 3, players make choices that can lock them out of certain dialogue paths. This dashboard shows the entire game text analysed, representing the full script, but a single player will never experience 100% of that text in one playthrough. The actual exposure rate is lower, even assuming 100% game completion.

Additionally, spaced repetition matters for vocabulary acquisition (Saksittanupab, 2024), and the list showing words appearing 12 or more times does not take that into account. A word appearing evenly throughout the entire story is more likely to be acquired than a word appearing 12 times in one cutscene.

On top of all this, some video games, especially old ones, used in-game dialogue to represent tutorials or mundane video game text - such as the boys in Link's Awakening telling you to save the game - if

the games analysed here utilised this mechanic, then some out-of-world text may have slipped into the analysis.
An AI sentence classifier evaluates the literal text, whereas human language is full of metaphors, irony, historical meaning and sarcasm. This means a sentence can be classified as an A1 sentence, while actually requiring far more linguistic and cultural knowledge to fully grasp.
Importantly, CEFR frameworks generally assign a universal difficulty level to words, but language acquisition is heavily influenced by the learner's native language, as shown by the FSI language difficulty list. That means that for a French-national learner all the English words derived from French may be easy and already understandable, even if they're formally at the C2 level, whereas a Japanese national may struggle way more than that with even B2 vocabulary.
At the same time though, these limitations highlight certain areas of improvement. The first one that comes to mind is making use of a more sophisticated slang filter to unify the analysis of all the games. Then, another area of improvement that may combat the issue with classifying sentences is simply displaying a random sample of sentences, alongside their classifications, to the user - perhaps only ones that are above 5 words to not have the user see sentences like "I agree.", "Yes" or "I feel good". What might also have to be done is to run the analysis once again, but this time without the word CEFR flattening, in order to get a comparable result. Yet another area of improvement is implementing a dialect converter, which, for example, converts all British and Australian words into their American English counterparts (or into British English). Using a video game trained sentence classifier is also an obvious improvement, however, as it stands, no such classifier exists, and creating one would require both a team of AI experts and linguists working in tandem. An idea that could be implemented is checking how often each word appears in the game, which could improve the acquisitional index score by accounting for the spread of the word in the game. What's more, the video games with non-essential quests should be analysed more thoroughly to get a list of only the necessary dialogues one must see in order to finish the game, without completing any side content. The most important area of improvement, however, is expanding the dataset to include more games that people want to play - after all, even the most methodologically sound video game dashboard is utterly useless for someone who can't find games they want to play in it.

All video game titles, cover art, character names, and dialogue data referenced in this project are the property of their respective copyright holders and are sourced, analyzed, and displayed strictly for non-commercial educational and scientific research purposes, qualifying as Fair Use under applicable copyright law

CITATIONS

Krashen, S. D. (1984). *Principles and practice in second language acquisition*. Oxford: Pergamon Press.

Niewiarowski, K. (2026). *"Press A to Confirm". Gry wideo jako narzędzie przyswajania języka angielskiego w oparciu o teorię Stephena Krashena. Wyniki badań pilotażowych* [In press]. Kraków: Wydawnictwo AGH.

Stephanie Rennick, Seán G. Roberts, (in press) The Video Game Dialogue Corpus. Corpora 19(1). preprint

Stephanie Rennick, Melanie Clinton, Elena Ioannidou, Liana Oh, Charlotte Clooney, E. T., Edward Healy, Seán G. Roberts (2023) Gender bias in video game dialogue. Royal Society Open Science 10(5). https://royalsocietypublishing.org/doi/10.1098/rsos.221095

Eleyan, Derar & Othman, Abed & Eleyan, Amna. (2020). Enhancing Software Comments Readability Using Flesch Reading Ease Score. Information. 11. 1-25. 10.3390/info11090430.

Zhou, Shixiang & Jeong, Heejin & Green, Paul. (2017). How Consistent Are the Best-Known Readability Equations in Estimating the Readability of Design Standards?. IEEE Transactions on Professional Communication. PP. 1-15. 10.1109/TPC.2016.2635720.

Dale, E., & Chall, J. S. (1948). A Formula for Predicting Readability. *Educational Research Bulletin*, *27*(1), 11–28. http://www.jstor.org/stable/1473169

Chall, J. S., & Dale, E. (1995). *Readability revisited: The new Dale-Chall readability formula*. Brookline Books.

Figueras, N. (2012). The impact of the CEFR. *ELT Journal*, *66*(4), 477–485. https://doi.org/10.1093/elt/ccs037

Laufer, Batia. (1992). How Much Lexis is Necessary for Reading Comprehension?. 10.1007/978-1-349-12396-4_12.

Hu, M. & Nation, P. (2000). Unknown vocabulary density and reading comprehension. *Reading in a Foreign Language*, *13(1),* 403-430.

Wolf, M. (2002). *The medium of the video game.* University of Texas Press.

Shahiwala, S., & Rahul, D. R. (2025). A qualitative analysis of gamers' experiences with vocabulary acquisition: Insights from grounded theory. *System, 135,* 103864. https://doi.org/10.1016/j.system.2025.103864

Nation, Paul. (2014). How much input do you need to learn the most frequent 9,000 words?. Reading in a Foreign Language. 26. 1-16. 10.64152/10125/66881.

Mayer, R., & Sims, V. (1994). For whom is a picture worth a thousand words? Extensions of a dual-coding theory of multimedia learning. *Journal of Educational Psychology, 86,* 389–401. https://doi.org/10.1037/0022-0663.86.3.389

Johansson, V. (2008). Lexical diversity and lexical density in speech and writing: A developmental perspective. *Working Papers, 53,* 61-79.

Riguel, E. (2014). Phrasal verbs: Usage and acquisition. *Athens Journal of Philology*, *1*(2), 111–126. https://doi.org/10.30958/ajp.1-2-3

Richards, Brian. (1987). Type/Token Ratios: what do they really tell us?. *Journal of child language. 14.* 201-9. 10.1017/S0305000900012885.

Mccarthy, Philip & Jarvis, Scott. (2010). MTLD, vocd-D, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment. *Behavior research methods. 42.* 381-92. 10.3758/BRM.42.2.381.

Saksittanupab, P. (2024). Enhancing Vocabulary Acquisition and Retention: The Role of Spaced Repetition in Language Learning. *Journal of Modern Learning Development*, *9*(5), 205–215. retrieved from https://so06.tci-thaijo.org/index.php/jomld/article/view/273598

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. https://doi.org/10.5281/zenodo.1212303

Hengel, E. (2026). *Textatistic* (Version 0.0.1) [Computer software]. GitHub. https://github.com/erinhengel/Textatistic

Frens, J. (2026). *lexical-diversity* (Version 0.1.1) [Computer software]. GitHub. https://github.com/jennafrens/lexical_diversity

Maksym, B. (2026). *cefrpy* [Computer software]. GitHub. https://github.com/Maximax67/cefrpy

dksysd. (2024). *cefr-classifier* [Machine learning model]. Hugging Face. https://huggingface.co/dksysd/cefr-classifier

Python Software Foundation. (2026). *statistics — Mathematical statistics functions*. Python 3.12.3 documentation. https://docs.python.org/3/library/statistics.html

Python Software Foundation. (2026). *runpy — Locating and executing Python modules*. Python 3.12.3 documentation. https://docs.python.org/3/library/runpy.html

Google. (accessed 2026, May 5). *Active voice*. Google Developers. https://developers.google.com/tech-writing/one/active-voice

Grammarly. (accessed 2026, May 5). *Subordinate Clause: Definition, Examples, and Usage*. https://www.grammarly.com/blog/grammar/subordinate-clause/

Scribbr. (accessed 2026, May 5). *Conditional Sentences | Definition, Examples & Forms*. https://www.scribbr.com/verbs/conditional-sentences/

Foreign Service Institute. (n.d.). *Foreign Language Training: Language Difficulty Rankings*. U.S. Department of State.

Internet Archive. (2026). *Wayback Machine*. https://archive.org/

Niewiarowski, K. (2026). *The Language of Video Games:* [Source code]. GitHub. https://github.com/Kamillon125/the_language_of_video_games

Niewiarowski, K. (2026). *kniew* [Source code]. GitHub. https://github.com/Kamillon125/kniew